

DriveShare

Project Team Plan — 3 Teams

ASP.NET Core · React · SQL Server · SignalR

فريق A Auth & Admin	فريق B Cars & Search	فريق C Booking & Notifications
-------------------------------	--------------------------------	--

1. Overview

للبيانات قاعدة من كاملة feature عن ومسؤول Frontend، وعضو Backend عضو فيه فريق كل — فرق 3 على مقسم المشروع حد بيستنى حد ومفيش تقريباً مستقل بشكل يشتغل فريق كل إن الهدف UI.

الفريق	Feature	Backend (.NET)	Frontend (React)
A فريق	Auth & Admin	Auth API, JWT, Admin endpoints	Login/Register pages, Admin dashboard
B فريق	Cars & Search	Cars CRUD, Search/Filter, Availability	Browse, Search, Car details, Owner dashboard
C فريق	Booking & Notif.	Rental API, SignalR Hub, Ratings	Booking flow, Dashboards, Notifications, Ratings

Auth الـ على بتعتمد الفرق باقي لأن الأهم الفريق ده. الأيمن ولوحة، الصلاحيات، الدخول تسجيل، بالمستخدمين متعلقة حاجة كل عن مسؤول منه هيطلع اللي Middleware

Backend (ASP.NET Core)

- ▶ Project setup كامل: Program.cs, EF Core, SQL Server connection string, Swagger
- ▶ DB Models للمشروع الكاملة: User, CarPost, RentalRequest, Rating, Notification + DbContext
- ▶ Initial Migration الـ نفس يستخدموا C و B فريق — الجداول كل تشمل Migration
- ▶ JWT Authentication: Register, Login, Logout, Refresh Token
- ▶ Role-based Authorization Middleware (Admin / CarOwner / Renter)
- ▶ AdminController + AdminService: الـ حسابات رفض/قبول Owners
- ▶ Admin: نشرها قبل السيارات إعلانات رفض/قبول
- ▶ Driver License: upload endpoint + الحالة تغيير (pending/verified/rejected)

API Endpoints:

```
POST    /api/auth/register
POST    /api/auth/login
POST    /api/auth/logout
GET     /api/auth/me
PUT     /api/admin/users/{id}/approve
PUT     /api/admin/users/{id}/reject
GET     /api/admin/users/pending
PUT     /api/admin/cars/{id}/approve
PUT     /api/admin/cars/{id}/reject
GET     /api/admin/cars/pending
POST    /api/license/upload
```

Frontend (React)

- ▶ Register: 3 role الـ حسب مختلفة نماذج (Admin / Owner / Renter) صفحة
- ▶ token للـ Axios interceptor + localStorage في JWT storage مع Login صفحة
- ▶ Auth Context أو Redux Slice: المشروع كل في ويوفرها المستخدم بيانات يحفظ
- ▶ Protected Routes: الـ role الـ من بيتحقق كل route
- ▶ Admin Dashboard: المعلقة Owners الـ حسابات جدول
- ▶ Admin: رفض/قبول وأزرار مع المعلقة السيارات إعلانات قائمة
- ▶ الرفع قبل الصورة preview مع Renter للـ القيادة رخصة رفع صفحة

Pages / Components:

```
/login          ← الدخول تسجيل صفحة
/register       ← التسجيل صفحة
/admin/users    ← Admin: إداره
/admin/cars     ← Admin: الإعلانات مراجعة
/profile/license ← القيادة رخصة رفع
components/ProtectedRoute ← Higher Order Component
context/AuthContext ← Auth state management
```

⚡ `authMiddleware` is a function that takes a `req` object and returns a `Promise`. It checks if the user is logged in by looking for a `JWT token` in the `req` object. If the token is not found, it returns a `401` status code. If the token is found, it checks if the token is valid by using the `jwt.verify` function. If the token is valid, it returns the `req` object. If the token is not valid, it returns a `401` status code. — `authMiddleware` is a function that takes a `req` object and returns a `Promise`. It checks if the user is logged in by looking for a `JWT token` in the `req` object. If the token is not found, it returns a `401` status code. If the token is found, it checks if the token is valid by using the `jwt.verify` function. If the token is valid, it returns the `req` object. If the token is not valid, it returns a `401` status code.

بيشوي +توفيق Cars & Search — B فريق

اللي المنصة قلب ده .التقويم وعرض والفلتره البحث لحد ،وحذفه وتعديله الإعلان إنشاء من — كامله السيارات إعلانات عن مسؤول حاجة أكثر معاه بيتعامل Renter

Backend (ASP.NET Core)

- CarController + CarService: CRUD كامل (Create, Read, Update, Delete)
- Business Rule: دي السيارة على نشط حجز فيه لو الإعلان حذف منع
- Search & Filter: Query Parameters (carType, brand, location, minPrice, maxPrice)
- Pagination (pageNumber, pageSize) السيارات قائمه على
- AvailabilityController: إضافة وجلب
- Car post status flow: pending → approved → rented عند الحجز بعد الإنشاء بعد الموافقة بعد الأيمن موافقة بعد
- بالإعلان وربطها الصور لرفع endpoint: السيارة صور

API Endpoints:

GET	/api/cars	← Browse + Search + Filter
GET	/api/cars/{id}	← واحدة سيارة تفاصيل
POST	/api/cars	← إعلان إنشاء (Owner only)
PUT	/api/cars/{id}	← إعلان تعديل
DELETE	/api/cars/{id}	← (محجوزة لو ممنوع) حذف
GET	/api/cars/{id}/availability	← المتاحة التواريخ
POST	/api/cars/{id}/availability	← تواريخ إضافة
POST	/api/cars/{id}/images	← صور رفع

Frontend (React)

- يقفها حد أي — login بدون تشتغل السيارات كاردات شبكة Browse: صفحة
- Search Bar + Filters: السعر نطاق ،الموقع ،الماركة ،السيارة نوع بـ فلتره
- Availability Calendar + صور + الإعلان معلومات كل :السيارة تفاصيل صفحة
- Availability Calendar: مختلفة بألوان والمحجوزة المتاحة التواريخ يعرض
- Owner Dashboard: وحذف تعديل أزرار مع إعلاناته قائمه
- صور رفع + المطلوبة الحقول كل :الإعلان تعديل/إنشاء نموذج
- badge (Pending / Approved / Rented) يوضح الإعلان حالة

Pages / Components:

/cars	← Browse page (public)
/cars/{id}	← Car details page
/owner/cars	← Owner dashboard
/owner/cars/new	← Create new listing
/owner/cars/{id}/edit	← Edit listing
components/CarCard	← Reusable car card
components/CarFilters	← Filter sidebar
components/AvailCalendar	← Availability calendar

⚡ B [Authorize] middleware A. mock JWT token .

شادي + أسامه — Booking & Notifications فريق C

بعد للتقييم، SignalR، بـ الفورية للإشعارات، Owner الـ رفض/لقبول، Renter الـ طلب من — كاملاً الحجز عملية سير عن مسؤول المشروع في جزء أصعب ده. الإيجار انتهاء

Backend (ASP.NET Core)

- ▶ RentalController + RentalService: محددة بتواريخ حجز طلب
- ▶ Owner: تلقائياً دي التواريخ في "Rented" لـ السيارة حالة تحديث → الطلب قبول
- ▶ Owner: فوراً Renter الـ إبلاغ → الطلب رفض
- ▶ حجز طلب بإرسال السماح قبل القيادة رخصة من التحقق
- ▶ SignalR Hub (/hubs/notifications): فورية إشعارات — Owner لـ جديد طلب — Renter لـ رفض/قبول
- ▶ NotificationController: مقروء ك تعليم + المستخدم إشعارات سجل جلب
- ▶ RatingController + RatingService: تقييم (1-5) + تعليق — فقط متاح — الإيجار اكتمال بعد فقط متاح
- ▶ مرة من أكثر الإيجار نفس يقيم يقدر مش Renter: حماية

API Endpoints:

POST	/api/rentals	جديد حجز طلب ←
GET	/api/rentals/my	← (Renter) طلباتي
GET	/api/rentals/incoming	← (Owner) الواردة الطلبات
PUT	/api/rentals/{id}/accept	← الطلب قبول
PUT	/api/rentals/{id}/reject	← الطلب رفض
GET	/api/rentals/{id}	← طلب تفاصيل
POST	/api/ratings	← تقييم إضافة
GET	/api/cars/{id}/ratings	← سيارة تقييمات
GET	/api/notifications	← الإشعارات سجل
PUT	/api/notifications/read	← كمقروء تعليم
WS	/hubs/notifications	← SignalR Hub

Frontend (React)

- ▶ الإجمالي السعر عرض + الكالندر من التواريخ اختيار: الحجز طلب صفحة
- ▶ أولاً يرفعها المستخدم من يطلب موجودة مش لو: القيادة رخصة من التحقق
- ▶ Renter Dashboard: الحالة مع طلباته قائمة (Pending / Accepted / Rejected / Completed)
- ▶ Owner Dashboard: طلب لكل رفض/قبول أزرار مع الواردة الطلبات قائمة
- ▶ Toast فوراً ك ويعرضها الإشعارات يستقبل — دائم اتصال: Socket.io Client (SignalR)
- ▶ Notification Bell: الموقرة غير الإشعارات عداد مع Navbar الـ في أيقونة
- ▶ نصي تعليق + نجوم 5 — فقط الإيجار اكتمال بعد يظهر: التقييم نموذج
- ▶ السيارة تفاصيل صفحة على التقييمات عرض

Pages / Components:

/booking/{carId}	← الحجز طلب صفحة
/renter/bookings	← Renter dashboard
/owner/bookings	← Owner: الواردة الطلبات
components/NotifBell	← الإشعارات أيقونة
components/RatingForm	← التقييم نموذج
components/BookingCard	← الطلب كارد
hooks/useSignalR	← Custom hook للإشعارات

⚡ `auth` `C` `Auth` `A` `car_id` `B`. `mock data` `mock data` `mock data`.

5. Timeline

	١ أسبوع	٢ أسبوع	٣ أسبوع	٤+ أسبوع
A فريق	Project setup + DB Models + Auth API	Admin endpoints + Swagger	Support + bug fixes	Testing + polish
B فريق	Cars Models + أساسي CRUD API	Search/Filter + Availability + Frontend Browse	Owner Dashboard + Integration مع A	Testing + polish
C فريق	Rental Models + أساسي Rental API	Frontend Booking flow + Dashboards	SignalR + Notifications + Ratings	Testing + كامل Integration

6. Shared Rules — الأول اليوم من عليها اتفاقوا

- DTOs / مجلد في Request DTO و Response DTO له endpoint كل — مشترك DTOs
- Unified Response: { success, data, message, statusCode } ترجع الـ APIs الـ كل
- Git Branches — review بدون main على merge مفيش، branch له فريق كل
- Swagger يتوثق كل endpoint، الأول اليوم من يضبطه A فريق — إجباري Swagger
- ⚡ الثاني اليوم في Git على Auth Middleware + mock JWT ينشر A فريق
- ثمانية migrations يعملوش ما C و B فريق — الكاملة DB Migrations يعمل A فريق
- Backend الـ مستني مش — أسبوع أول mock data يستخدم فريق كل Frontend
- ✓ merge الـ قبل Service لكل الأقل على واحد unit test يكتب فريق كل

7. Unified Response + Project Structure

API Response Wrapper (C#)

بيستخدمه والكل يوم أول في بيعمله A فريق — ده الشكل نفس ترجع الـ APIs الـ كل

```
public class ApiResponse<T>
{
    public bool    Success    { get; set; }
    public string  Message    { get; set; }
    public T       Data       { get; set; }
    public int     StatusCode { get; set; }
}
```

Project Folder Structure

```
DriveShare.API/
├── Controllers/
│   ├── AuthController.cs      ← فريق A
│   ├── AdminController.cs     ← فريق A
│   ├── CarController.cs       ← فريق B
│   ├── RentalController.cs    ← فريق C
│   ├── RatingController.cs    ← فريق C
│   └── NotificationController ← فريق C
├── Services/
│   ├── AuthService.cs         ← فريق A
│   └── CarService.cs           ← فريق B
```

```
| └─ RentalService.cs      ← فريق C
| └─ Models/              ← كلها ينشئها A فريق
|   └─ User.cs
|   └─ CarPost.cs
|   └─ RentalRequest.cs
|   └─ Rating.cs
|   └─ Notification.cs
| └─ DTOs/                ← هنا يضيف الكل - مشترك
|   └─ Auth/
|   └─ Cars/
|   └─ Rentals/
| └─ Hubs/
|   └─ NotificationHub.cs ← فريق C
| └─ Data/
|   └─ AppDbContext.cs    ← فريق A
| └─ Migrations/          ← فقط A فريق
| └─ Program.cs           ← فريق A
```